

CloudShield AI

31 — Observability & Logging Specification

Version 1.0 | Product-Level Observability, Telemetry & Audit Specification

CloudShield AI / CloudSpartanX

Document purpose: Define the observability architecture, logging standards, metrics, traces, events, audit telemetry, dashboards, alerts, health signals, retention, privacy, security, correlation, SLOs, and operational response requirements for CloudShield AI.

Canonical principle: If CloudShield AI cannot explain what happened, why it happened, what state it reached, and what evidence supports that state, the operation is not operationally trustworthy.

1. Purpose and Scope

This specification defines observability for the entire CloudShield AI product and its DAIL subsystem. It covers product services, cloud integrations, AI/LLM operations, asynchronous workers, verification, promotion, evidence, external integrations, and infrastructure.

- Define logs, metrics, traces, domain events, audit events, and evidence telemetry.
- Define correlation across API requests, asynchronous jobs, cloud operations, AI calls, DAIL verification, and promotion.
- Define structured logging and sensitive-data handling.
- Define service health, SLO, alerting, and dashboard requirements.
- Define observability for AWS infrastructure and Terraform-managed resources.
- Define security and audit telemetry.
- Define AI-specific and DAIL-specific telemetry.
- Define retention, access, cost, sampling, and privacy controls.
- Define incident investigation and operational readiness requirements.

2. Observability Model

CloudShield AI uses five complementary telemetry classes.

Telemetry	Primary Question	Examples
Logs	What happened?	Errors, lifecycle transitions, provider failures.
Metrics	How often/how much/how fast?	Latency, queue depth, failure rate.
Traces	How did a request flow?	API →service →worker →AWS/LLM.
Domain Events	What meaningful state change occurred?	Finding created, candidate verified.
Audit/Evidence	Who/what authorized and proved it?	Approval, promotion, verification evidence.

Logs explain execution; metrics explain system behavior; traces explain causality; domain events explain business state; audit/evidence establish accountability and proof.

3. Observability Architecture

Recommended logical flow:

Application/Workers/AWS/LLM/External Integrations → Telemetry SDK/Agent → Collection Layer →
Logs/Metrics/Traces/Event Stores → Dashboards/Alerts/SIEM/Operational Tools.

Source	Signals
API	Request logs, latency metrics, traces, authorization/security events.
Workers	Job lifecycle, duration, retries, failures, queue metrics.
Cloud Integration	Provider calls, throttling, authorization failures, reconciliation.
AI Layer	Model request metadata, latency, token/cost usage, safety outcomes.
DAIL	Verification checks, invalidations, promotion blocks, execution evidence.
Database	Connections, latency, errors, saturation, backup status.
Queue	Depth, age, retries, DLQ count.
Infrastructure	CPU/memory, restarts, network, storage, health.
External integrations	Delivery latency, failures, rate limits, auth health.

4. Core Observability Principles

- Structured machine-readable telemetry is the default.
- Every important operation has a correlation identifier.
- Tenant context is available to authorized telemetry consumers but minimized in broadly accessible logs.
- Secrets and credentials are never logged.
- Observability must not materially alter system behavior or become a security bypass.
- High-value security and lifecycle events receive stronger durability than debug logs.
- Metrics are low-cardinality; detailed identifiers belong in logs/traces/events rather than metric labels.
- Observability failures must not cause unsafe cloud mutation.
- Telemetry must distinguish stale, partial, unknown, failed, and successful state.
- Audit records are not equivalent to ordinary operational logs.

5. Correlation and Trace Context

Field	Purpose
request_id	Unique HTTP/API request identifier.
correlation_id	End-to-end workflow correlation.
trace_id	Distributed trace identifier.
span_id	Individual operation span.
job_id	Asynchronous job identifier.
tenant_id	Tenant context, protected.
actor_id	Authenticated actor/service reference, protected.
resource_id	Affected CloudShield resource.

Field	Purpose
candidate_version_id	Exact remediation candidate version.
verification_run_id	DAIL verification reference.
promotion_id	Governed promotion reference.
integration_id	External integration reference.

Correlation IDs should propagate across API →queue →worker →provider/LLM call →database/evidence operations where technically practical.

6. Structured Log Schema

Field	Requirement
timestamp	UTC event timestamp.
level	DEBUG/INFO/WARN/ERROR/FATAL.
service	Producing service.
environment	dev/test/staging/prod.
event_name	Stable machine-readable event name.
message	Safe human-readable summary.
request_id	Request correlation.
correlation_id	Workflow correlation.
trace_id	Distributed trace.
tenant_ref	Protected tenant reference when appropriate.
actor_ref	Protected actor/service reference.
resource_type/id	Affected resource.
operation	Operation class.
status	Result/state.
duration_ms	Elapsed time when relevant.
error_code	Stable error class.
retryable	Boolean where relevant.
provider	External provider if applicable.

Additional fields are allowed when documented. Avoid arbitrary high-cardinality fields that make logs difficult to query or increase cost unnecessarily.

7. Log Levels

Level	Use
DEBUG	Detailed diagnostic information; normally disabled or sampled in production.
INFO	Normal meaningful lifecycle/operational events.
WARN	Unexpected but recoverable condition or degradation.
ERROR	Operation failed or requires attention.
FATAL	Process/service cannot continue safely.

- Do not use ERROR for ordinary expected rejection such as normal authorization denial unless it represents a security anomaly.
- Security events use dedicated event classifications in addition to log severity.
- Production DEBUG logging must be controlled, temporary, and audited where it could expose sensitive context.

8. What Must Never Be Logged

- Cloud provider access keys or secret keys.
- Temporary credentials or session tokens.
- OAuth refresh tokens.
- Passwords or private keys.
- Raw authentication headers.
- Unredacted webhook secrets.
- Full customer evidence when not required for the log event.
- Sensitive AI prompt/context payloads by default.
- Raw LLM responses containing customer data unless explicitly protected as evidence.
- Secrets embedded in Terraform state or configuration.
- Unnecessary personally identifiable information.

9. Data Redaction and Minimization

Data	Treatment
Credentials	Always redact/drop.
Authorization tokens	Always redact/drop.
Customer identifiers	Use protected references where possible.
Cloud resource names	Log only when operationally necessary.
AI context	Log metadata/hash/reference, not full content by default.
Evidence content	Reference artifact ID; avoid duplicating payload.
External tickets/comments	Reference IDs; do not copy full content into logs.
Request bodies	Do not log by default; allow-list safe fields.
Error payloads	Sanitize provider responses before logging.

10. Core Service Metrics

Metric	Purpose
api_requests_total	Request volume.
api_request_duration_ms	API latency.
api_errors_total	API failures by stable class.
authorization_denials_total	Access-control failures.
jobs_started_total	Async workload volume.
jobs_completed_total	Job throughput.
jobs_failed_total	Worker failure rate.
job_duration_ms	Processing duration.
queue_depth	Backlog.
queue_oldest_age	Backlog freshness.
dlq_messages_total	Poison/failure workload.
db_query_duration_ms	Database performance.
db_connections	Database saturation.
external_request_duration_ms	Provider/integration latency.
external_errors_total	External dependency failures.

11. Cloud Integration Metrics

Metric	Dimensions
cloud_discovery_resources_total	provider, resource_type, result.
cloud_discovery_duration_ms	provider, scope_class.
cloud_provider_requests_total	provider, operation_class, result.
cloud_provider_throttles_total	provider, operation_class.
cloud_permission_denials_total	provider, operation_class.
cloud_reconciliation_required_total	provider, operation_class.
cloud_mutations_total	provider, operation_class, result.
cloud_observation_freshness_seconds	provider, resource_class.

Dimensions must remain low-cardinality. Individual tenant/resource IDs should not normally be metric labels.

12. Security and Finding Metrics

Metric	Purpose
findings_created_total	New findings.
findings_resolved_total	Resolved findings.
findings_by_severity	Severity distribution.
risk_assessments_total	Risk calculation volume.
unknown_security_results_total	Rules lacking sufficient evidence.
stale_security_context_total	Analysis using stale context.
security_rule_failures_total	Rule-engine failures.
security_events_total	Security event volume.

The product must distinguish 'no finding detected' from 'could not evaluate.'

13. AI/LLM Observability

Metric/Event	Purpose
ai_requests_total	AI workload volume.
ai_request_duration_ms	Model latency.
ai_provider_errors_total	Provider failures.
ai_rate_limits_total	Provider quota/rate-limit pressure.
ai_tokens_total	Usage/cost estimation.
ai_cost_estimate	Budget monitoring.
ai_output_validation_failures_total	Malformed/unsafe model outputs.
ai_guardrail_blocks_total	Safety blocks.
ai_recommendation_acceptance	Downstream usefulness.
ai_model_version	Evaluation and reproducibility metadata.

- Record logical model/provider identifiers rather than exposing secret provider configuration.
- AI telemetry must support evaluation without requiring storage of raw sensitive prompts.
- Model/version/prompt-policy references should be recorded for reproducibility.
- AI latency and cost must be observable independently from domain operation success.

14. DAIL Observability

Metric/Event	Meaning
dail_verification_started_total	Verification workload.
dail_verification_completed_total	Verification completion.
dail_verification_failed_total	Failed verification.
dail_check_failed_total	Individual deterministic check failures.
dail_promotion_blocked_total	Promotion blocked by gate.
dail_promotion_started_total	Promotion execution started.
dail_promotion_completed_total	Promotion completed.
dail_reconciliation_required_total	Ambiguous external outcome.
dail_invalidated_total	Previously valid state invalidated.
dail_verifier_version	Exact verifier implementation.

DAIL telemetry must make it possible to answer: which candidate version was checked, by which verifier version, against what context, with what result, and why promotion was or was not allowed.

15. Lifecycle Event Taxonomy

Event	Meaning
connection.created	Cloud/external connection created.
connection.validated	Connection validation completed.
discovery.started	Discovery started.
discovery.completed	Discovery completed.
discovery.partial	Discovery completed partially.
finding.created	Finding created.
risk.assessed	Risk assessment completed.
candidate.created	Candidate created.
candidate.versioned	Candidate version created.
impact.completed	Impact analysis completed.
verification.started	Verification started.
verification.completed	Verification completed.
verification.failed	Verification failed.
approval.required	Governance approval required.
approval.granted	Approval granted.
approval.rejected	Approval rejected.
promotion.started	Promotion started.
promotion.completed	Promotion completed.
promotion.failed	Promotion failed.
promotion.reconciliation_required	Outcome ambiguous.
trusted_state.changed	Trusted state changed.
drift.detected	Observed state diverged from trusted state.
integration.failed	External integration failure.

16. Audit Event Model

Audit events are durable records of security-sensitive and governance-sensitive actions. They are distinct from ordinary application logs.

Field	Purpose
audit_event_id	Stable audit ID.

Field	Purpose
timestamp	UTC event time.
actor	Human/service identity.
tenant	Tenant context.
action	Action performed.
target	Target resource.
before_state	Relevant prior state reference.
after_state	Relevant resulting state reference.
candidate_version	Exact candidate where applicable.
verification_run	Verification reference.
approval	Approval reference.
promotion	Promotion reference.
result	Success/failure/block.
reason	Safe reason code.
correlation_id	Workflow trace.

- Audit records must be tamper-resistant according to the evidence/audit architecture.
- Audit events must be access-controlled.
- Audit event generation must not depend solely on client-provided data.
- Sensitive payloads are referenced rather than duplicated.

17. Evidence vs Audit vs Logs

Artifact	Authority/Purpose
Operational Log	Diagnostic execution detail.
Metric	Aggregate operational measurement.
Trace	Distributed execution path.
Domain Event	Meaningful product state transition.
Audit Event	Accountability/security/governance record.
Evidence Artifact	Proof supporting a finding, verification, or promotion.

No single telemetry class should be treated as a substitute for all others.

18. Distributed Tracing

- Trace inbound API requests.
- Create spans for database, queue, cloud provider, LLM provider, external integration, and object-storage operations where supported.
- Propagate trace context through asynchronous job metadata.
- Do not put secrets or raw customer payloads into span attributes.
- Use sampling in high-volume paths while retaining 100% of critical promotion/error traces where operationally feasible.
- Trace IDs must be searchable from logs and audit records.

19. Asynchronous Job Observability

Job Signal	Requirement
Queued	Timestamp and queue reference.
Started	Worker/service and attempt.

Job Signal	Requirement
Heartbeat	For long-running work where required.
Retry	Attempt number and reason.
Progress	Optional bounded progress metric.
Succeeded	Completion duration/result.
Failed	Stable failure code and retryability.
Reconciliation	Ambiguous external outcome reference.
Cancelled	Cancellation actor/reason.

Job telemetry must allow operators to distinguish queue delay from execution delay.

20. Health Checks

Health Type	Meaning
Liveness	Process can continue running.
Readiness	Service can accept work.
Dependency health	Required dependencies available.
Integration health	External connection usable.
Worker health	Worker can acquire/process jobs.
Promotion health	Safety-critical dependencies and authorization path operational.

A service may be alive but not ready. Health endpoints must not expose sensitive dependency details publicly.

21. SLO/SLA Baseline

Area	Initial SLO Target
API availability	$\geq 99.5\%$ monthly for MVP, excluding planned maintenance.
Read API latency	$p95 \leq 500$ ms for normal single-resource requests.
Mutating API latency	$p95 \leq 1$ s to durable acceptance for async operations.
Job acceptance	$\geq 99.9\%$ of valid jobs durably accepted.
Critical promotion path	No silent failure; every accepted promotion reaches terminal/reconciliation state.
Telemetry delivery	$\geq 99\%$ of critical audit/security events durably recorded.
Discovery freshness	Configured per connection/resource class; no universal fixed freshness assumption.

These are engineering starting targets, not customer contractual SLAs. Production commitments must be finalized using measured workload and business requirements.

22. Alerting Philosophy

- Alert on symptoms requiring action, not every error.
- Separate critical security/promotion alerts from ordinary application alerts.
- Alerts should have owner, severity, condition, runbook, and expected response.
- Use multi-signal correlation to reduce noisy alerts.
- Never suppress alerts for safety-critical verification/promotion failures solely to reduce noise.

- Alerts should identify whether the issue affects availability, correctness, security, freshness, or governance.

23. Critical Alerts

Alert	Severity	Trigger Example
API availability degraded	Critical	Sustained 5xx/availability breach.
Database unavailable	Critical	Required DB connectivity lost.
Promotion failure spike	Critical	Unusual failure rate or blocked critical workflow.
Reconciliation backlog	Critical	Ambiguous cloud outcomes exceed threshold.
Audit delivery failure	Critical	Critical audit events cannot be durably recorded.
Credential/auth failure spike	High	Repeated cloud/integration authentication failures.
Queue backlog	High	Oldest job exceeds operational threshold.
DLQ growth	High	Persistent failed jobs.
AI provider outage	Medium/High	Provider unavailable beyond tolerance.
Telemetry pipeline failure	High	Critical observability path unavailable.
Unexpected privileged activity	High/Critical	Suspicious promotion/admin behavior.

24. Dashboards

Dashboard	Primary Audience	Contents
Executive Product Health	Leadership	Availability, findings, risk, trusted state, major incidents.
Platform Operations	SRE/engineering	API, DB, queues, workers, infrastructure.
Security Operations	Security team	Security events, auth failures, suspicious activity.
Cloud Integrations	Cloud/infrastructure team	Discovery, provider errors, throttling, freshness.
AI Operations	AI/engineering	Latency, errors, usage, cost, guardrails.
DAIL Safety	Platform/security	Verification, invalidation, blocked promotion, reconciliation.
External Integrations	Integration owners	Webhook/ticket/SCM/CI delivery health.

25. Tenant-Level Observability

- Tenant-level dashboards may expose product health and usage to authorized tenant administrators.
- One tenant must not see another tenant's operational data.
- Global operator telemetry should use protected tenant references and access controls.
- Tenant usage metrics can support quotas and cost attribution.
- Do not expose internal infrastructure topology unnecessarily to customers.

26. Observability Access Control

Role	Access
Tenant User	Product-facing health/status only as permitted.
Tenant Admin	Tenant integration/job/audit views.
Security Operator	Security events and relevant audit/evidence.
SRE/Platform Operator	Infrastructure and service telemetry.
AI Operator	AI telemetry and evaluation data.
DAIL Operator	Verification/promotion telemetry and evidence.
Auditor	Read-only authorized audit/evidence.
Developer	Non-production detailed telemetry; production access controlled.

27. Retention

Telemetry	Indicative Retention
Debug logs	Short, e.g. 3–7 days.
Application logs	Medium, e.g. 14–30 days.
Security logs	Longer, policy-driven.
Audit events	Long-term, policy-driven.
Evidence	Per evidence/tenant retention policy.
Metrics	Longer aggregated retention.
Traces	Short/medium with critical traces retained longer where useful.
DLQ/job metadata	Until operationally resolved + retention window.

Exact retention values must be finalized against compliance, customer contracts, storage cost, and incident-investigation requirements.

28. Sampling

- High-volume successful API traces may be sampled.
- Errors and critical state transitions should receive elevated sampling.
- Promotion, verification failure, security events, and reconciliation events should be retained at high fidelity.
- Sampling must not remove required audit/evidence records.
- Debug sampling must never become a substitute for deterministic audit.

29. Observability Cost Controls

- Use structured logs and bounded payload sizes.
- Avoid high-cardinality metric labels.
- Sample traces intelligently.
- Use lifecycle policies for logs and artifacts.
- Separate hot operational telemetry from cold audit/evidence storage.
- Monitor telemetry ingestion cost.
- Prevent noisy integrations or tenants from overwhelming shared telemetry.

30. Security Monitoring

Signal	Detection Goal
Authentication failures	Credential abuse/account attack.
Authorization denials	Privilege probing.
Promotion anomalies	Unexpected high-impact actions.
Cloud role assumption	Unexpected customer-cloud access.
Webhook failures/signature failures	Integration attack or misconfiguration.
AI guardrail blocks	Potential prompt injection/unsafe request.
Secret access anomalies	Potential credential compromise.
Terraform drift	Unauthorized infrastructure change.
Evidence access	Sensitive data access monitoring.

31. Incident Correlation

An incident investigation should be able to start from any major identifier and move through the control loop.

request_id ↔ correlation_id ↔ job_id ↔ resource_id ↔ candidate_version_id ↔ verification_run_id ↔ approval_id ↔ promotion_id ↔ evidence_id ↔ audit_event_id

- Identifiers should be indexed/searchable where operationally justified.
- Cross-system correlation must include external event/ticket IDs when applicable.
- Incident responders should be able to reconstruct sequence and actor without relying on undocumented logs.

32. Drift and Trusted-State Observability

Event	Telemetry Requirement
Drift detected	Asset, trusted version/reference, observed version/time, source.
Trusted state invalidated	Reason, triggering observation/dependency change.
Reverification required	Candidate/context and required checks.
Promotion blocked	Gate and reason code.
Trusted state restored	Promotion/reconciliation evidence reference.

Trusted state must have observable provenance and a reconstructable history.

33. Terraform/AWS Infrastructure Observability

- Track infrastructure deployment success/failure.
- Track Terraform plan/apply events.
- Track infrastructure drift.
- Track critical IAM/network/storage/database changes.
- Correlate deployments with application versions.
- Record deployment actor/role and change reference.
- Alert on unexpected security-boundary changes.
- Do not expose Terraform state contents in ordinary telemetry.

34. External Integration Observability

Metric/Event	Purpose
integration_requests_total	Volume.
integration_failures_total	Failure rate.
integration_latency_ms	Provider performance.
integration_retries_total	Retry pressure.
integration_rate_limits_total	Provider throttling.
integration_auth_failures_total	Credential health.
integration_events_pending	Delivery backlog.
integration_event_age	Staleness.
integration_reconciliation_total	State mismatch/ambiguity.

35. Observability for Partial and Unknown State

- Use explicit metrics/events for PARTIAL, UNKNOWN, STALE, and INVALIDATED states.
- Do not count UNKNOWN security evaluations as successful evaluations.
- Dashboards should show incomplete coverage separately from clean results.
- Promotion dashboards must show blocked/uncertain context distinctly.
- Discovery completeness should be measurable per connection/scope/resource class.

36. Telemetry Reliability

- Critical audit/security events require durable delivery.
- Operational telemetry may degrade under load, but critical governance telemetry must have a stronger path.
- Telemetry failure must not block safe read operations unnecessarily.
- Telemetry failure must not permit unsafe mutation.
- Backpressure must be bounded.
- Telemetry queues require dead-letter/recovery handling where applicable.

37. Observability Testing

Test	Requirement
Schema tests	Telemetry conforms to documented schema.
Correlation tests	IDs propagate across synchronous/asynchronous boundaries.
Redaction tests	Secrets never appear in logs.
Metric tests	Counters/gauges emitted on required paths.
Audit tests	Critical actions create audit records.
Failure tests	Errors produce actionable telemetry.
Sampling tests	Critical events are not lost.
Alert tests	Critical alert conditions fire.
Recovery tests	Telemetry pipeline outage does not create unsafe state.
Tenant tests	Telemetry access respects tenant boundaries.

38. Operational Runbooks

Every critical alert must link to a runbook covering:

- Symptoms and scope.
- Likely causes.
- Queries/metrics to inspect.
- Relevant correlation identifiers.
- Immediate mitigation.
- Safety implications.
- Recovery procedure.
- Escalation owner.
- Evidence to preserve.
- Post-incident follow-up.

39. Incident Response Telemetry

- Preserve relevant logs, traces, audit events, and evidence before retention expiry.
- Record incident ID in relevant operational notes where supported.
- Do not alter evidence to hide incident activity.
- Use read-only investigation roles where possible.
- Security incidents involving customer-cloud access require provider-side audit correlation.
- Promotion incidents require candidate/version/verification/audit reconstruction.

40. MVP Observability Baseline

- Structured application/worker logs.
- Request/job/correlation IDs.
- Core API and worker metrics.
- Queue depth and DLQ metrics.
- Database health metrics.
- Cloud provider request/error/throttle metrics.
- AI latency/error/usage/guardrail metrics.
- DAIL verification/promotion telemetry.
- Durable audit events.
- Evidence references.
- Distributed tracing for critical paths.
- Core operational/security dashboards.
- Critical alerts and runbooks.
- Secret redaction.
- Tenant-aware access controls.
- Basic retention/lifecycle policies.

41. Implementation Sequence

1. Define canonical telemetry/event schema.
2. Implement correlation/request/job ID propagation.
3. Implement structured logging and redaction middleware.
4. Implement application/API metrics.
5. Implement worker/queue metrics.
6. Implement distributed tracing.

7. Implement cloud integration telemetry.
8. Implement AI/LLM telemetry.
9. Implement DAIL verification/promotion telemetry.
10. Implement durable audit events.
11. Implement dashboards.
12. Implement alerts and runbooks.
13. Implement retention and cost controls.
14. Add observability contract/security tests.
15. Validate incident reconstruction end-to-end.

42. Security Acceptance Criteria

16. No secrets or credentials appear in operational logs.
17. Critical audit events are durably recorded.
18. Tenant-restricted telemetry cannot cross tenant boundaries.
19. Promotion operations are fully traceable.
20. Verification runs identify exact candidate versions and verifier versions.
21. Cloud provider operations can be correlated to product workflows.
22. AI telemetry does not unnecessarily persist sensitive prompt/context data.
23. External webhook/security events are observable and auditable.
24. Terraform/infrastructure changes are traceable.
25. Critical alert conditions are tested.

43. Functional Acceptance Criteria

26. An operator can trace a user action from API request to resulting job/state.
27. An operator can identify why a job failed and whether it is retryable.
28. An operator can distinguish queue delay from worker execution delay.
29. An operator can identify stale/partial/unknown cloud observations.
30. An operator can reconstruct a remediation candidate's verification and promotion path.
31. An operator can identify the external provider operation responsible for a failure.
32. AI latency, failure, usage, and safety outcomes are measurable.
33. DAIL blocked promotions and reconciliation cases are visible.
34. Critical incidents have actionable alerts and runbooks.

44. Traceability

Source	Observability Dependency
00 — Master System Specification	Evidence, audit, trust, monitoring, end-to-end control loop.
01 — Software Requirements Specification	Observability, reliability, security, performance, audit.
02 — System Architecture Specification	Services, workers, queues, data stores, deployment.
03 — Functional Specification	Jobs, state transitions, verification, promotion, failure flows.
04 — Product & UX Specification	Status, stale/partial/error states, notifications.
05 — Domain & Data Model	Domain events, evidence, audit, identifiers.
06 — State & Lifecycle	Lifecycle transitions, invalidation, reconciliation.
07 — Identity & Dependency	Asset/dependency identity and change correlation.
08 — Security & Threat Model	Security monitoring, credential protection, audit.
09 — Intelligence & AI Architecture	AI telemetry, model/version evaluation.

Source	Observability Dependency
10 — LLM Integration	Provider latency, usage, failures, routing.
11 — AI Safety & Guardrails	Guardrail events, blocks, safety telemetry.
12–26 — DAIL subsystem	Verification, promotion, evidence, testing, DevOps telemetry.
27 — AWS & Terraform	Infrastructure logs, metrics, drift, IAM, deployment.
28 — API Specification	Request IDs, jobs, errors, API lifecycle operations.
29 — Cloud Integration	Provider calls, throttling, reconciliation, cloud health.
30 — External Integration	Webhook, ticket, SCM, CI/CD and integration telemetry.

45. Engineering Decisions

Decision	Rationale
Five telemetry classes	Prevents logs/metrics from being overloaded with audit/evidence responsibilities.
Structured logs	Improves search, automation, and incident reconstruction.
Correlation propagation	Connects distributed workflows.
Separate audit/evidence	Protects accountability and proof from operational log retention.
Low-cardinality metrics	Controls cost and preserves metric usability.
High-fidelity critical events	Protects promotion and security observability.
Explicit unknown/partial metrics	Prevents false-clean operational conclusions.
AI payload minimization	Reduces sensitive-data exposure.
DAIL-specific telemetry	Makes safety-critical verification explainable.
Runbook-linked alerts	Turns telemetry into operational action.

46. Definition of Done

- Canonical observability schema is implemented.
- Structured logs and redaction are active.
- Correlation IDs propagate through API and asynchronous workflows.
- Core metrics and dashboards are operational.
- Distributed tracing covers critical paths.
- Cloud, AI, DAIL, and external integration telemetry is implemented.
- Critical audit events are durable and access-controlled.
- Promotion and verification can be reconstructed end-to-end.
- Security events and infrastructure drift are observable.
- Critical alerts have tested runbooks.
- Retention, sampling, and cost controls are configured.
- Tenant isolation is enforced for telemetry access.
- Observability security/contract tests pass.
- Telemetry failure cannot bypass safety or promotion controls.

47. Glossary

Term	Definition
------	------------

Term	Definition
Observability	Ability to infer internal system state from emitted telemetry.
Log	Structured record describing an execution event.
Metric	Numeric measurement of system behavior.
Trace	Distributed representation of an operation across services.
Domain Event	Meaningful product state transition.
Audit Event	Durable accountability record for sensitive actions.
Evidence	Artifact supporting a security, verification, or promotion conclusion.
Correlation ID	Identifier linking related operations.
SLO	Service Level Objective.
DLQ	Dead-letter queue.
DAIL	Deterministic AI Infrastructure Lifecycle subsystem.
Trusted State	State earned through governed successful verification/promotion.